

Cinery ❖❖❖ M❖❖mo technique complet

- 📖 CINERY — Mémo technique complet
 - Table des matières
- 1. JavaScript ES6+
 - Variables
 - Arrow functions
 - Template literals (backticks)
 - Destructuring
 - Spread / Rest
 - Optional chaining `?.` et Nullish coalescing `??`
 - Modules (import/export)
 - Async/Await & Promises
 - Méthodes de tableaux essentielles
 - Set (utilisé dans MovieActions)
- 2. TypeScript
 - Types de base
 - Interfaces vs Type aliases
 - Union et intersection
 - Generics
 - Utility types (à retenir)
 - Enums
 - Type narrowing
 - `as` , `as const` , `satisfies`
- 3. React 19
 - Composant fonctionnel
 - JSX
 - `useState`
 - `useEffect`
 - `useTransition` (utilisé dans MovieActions)
 - `Suspense`
 - Différences Server vs Client Components
- 4. Next.js 16 (App Router)
 - Structure de fichiers
 - Routes dynamiques
 - Metadata (SEO)
 - Data fetching (fetch natif)
 - Server Actions
 - Navigation
 - Image optimisée
 - Middleware (le piège de Cinery)
- 5. Tailwind CSS 4
 - Concept : utility-first
 - Cheatsheet des classes courantes

- `@theme` (Tailwind 4)
- 6. PostgreSQL
 - Types courants
 - Table basique
 - Contraintes
 - Relations
 - Requêtes de base
 - Index
 - Transactions
- 7. Prisma 7 (ORM)
 - Schema (prisma/schema.prisma)
 - Attributs importants
 - Migrations
 - Queries
 - Opérateurs de filtre
 - Adapter PostgreSQL (Prisma 7)
- 8. Auth.js v5 & OAuth 2.0
 - OAuth 2.0 — le flow
 - Setup Auth.js v5
 - Session strategies
 - Utilisation
 - Tables requises par PrismaAdapter (noms exacts)
- 9. HTTP / REST / API
 - Méthodes HTTP
 - Codes de statut
 - Headers courants
 - Fetch API (native)
 - Bearer token (TMDB v4)
 - CORS (Cross-Origin Resource Sharing)
 - REST vs GraphQL
- 10. Git & GitHub
 - Commandes essentielles
 - .gitignore
 - Workflow typique
- 11. Vercel & Neon (déploiement)
 - Vercel
 - Neon
 - Runtime Node vs Edge
- 12. Node.js
 - Modules
 - Async/await + Promises
 - npm / package.json
 - Variables d'environnement
- 13. Sécurité web (OWASP Top 10)
 - 1. XSS (Cross-Site Scripting)
 - 2. CSRF (Cross-Site Request Forgery)
 - 3. SQL Injection
 - 4. IDOR (Insecure Direct Object Reference)

- [5. Authentication broken](#)
- [6. Exposition de secrets](#)
- [7. Autres vulnérabilités à connaître](#)
- [Headers de sécurité](#)
- [Cookies](#)
- [14. Terminal Bash](#)
 - [Navigation](#)
 - [Fichiers](#)
 - [Recherche](#)
 - [Redirections](#)
 - [Pipes](#)
 - [Autre](#)
- [15. Concepts transverses](#)
 - [Client-side vs Server-side rendering](#)
 - [Cache](#)
 - [Async / synchrone](#)
 - [Event loop \(Node.js\)](#)
 - [Rate limiting](#)
 - [Env : dev vs prod](#)
- 📌 [Points bonus à retenir](#)
 - [Le “je sais ce que je ne maîtrise pas bien”](#)
 - [Le vocabulaire pro](#)
 - [Les 3 questions pièges classiques](#)
- 📖 [Ressources pour approfondir](#)

📌 CINERY — Mémo technique complet

Bible d’entretien couvrant **toutes** les technologies utilisées dans le projet. À lire la veille d’un entretien, ou à consulter pendant.

Table des matières

1. [JavaScript ES6+](#)
2. [TypeScript](#)
3. [React 19](#)
4. [Next.js 16 \(App Router\)](#)
5. [Tailwind CSS 4](#)
6. [PostgreSQL](#)
7. [Prisma 7 \(ORM\)](#)
8. [Auth.js v5 & OAuth 2.0](#)
9. [HTTP / REST / API](#)
10. [Git & GitHub](#)
11. [Vercel & Neon \(déploiement\)](#)
12. [Node.js](#)
13. [Sécurité web \(OWASP\)](#)
14. [Terminal Bash](#)

1. JavaScript ES6+

Variables

```
var x = 1;           // Ancienne syntaxe, scope fonction, hoisted (à éviter)
let y = 2;           // Scope bloc, réassignable
const z = 3;         // Scope bloc, non réassignable (mais objets modifiables)
```

À dire : “En 2025 on n'utilise plus `var`. `const` par défaut, `let` seulement si on doit réassigner.”

Arrow functions

```
// Fonction classique
function add(a, b) { return a + b; }

// Arrow function
const add = (a, b) => a + b;

// Arrow avec plusieurs lignes
const add = (a, b) => {
  const sum = a + b;
  return sum;
};
```

Différence clé : les arrow functions n'ont pas leur propre `this`, elles héritent du contexte parent. Utile dans les callbacks.

Template literals (backticks)

```
const name = "Rayane";
const msg = `Bonjour ${name}, tu as ${1 + 1} messages.`;
// "Bonjour Rayane, tu as 2 messages."
```

Destructuring

```

// Objet
const { name, age } = { name: "Rayane", age: 20, city: "Rouen" };
// name = "Rayane", age = 20

// Renommer
const { name: firstName } = { name: "Rayane" };
// firstName = "Rayane"

// Défaut
const { role = "user" } = {}; // role = "user"

// Tableau
const [first, second] = [1, 2, 3];
// first = 1, second = 2

```

Spread / Rest

```

// Spread (étaier)
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }

// Rest (grouper)
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}

sum(1, 2, 3, 4); // 10

```

Optional chaining `?.` et Nullish coalescing `??`

```

const user = { profile: null };

user.profile.name; // ✖ TypeError: Cannot read properties of null
user.profile?.name; // ✔ undefined (pas d'erreur)

const name = user.name ?? "Invité"; // "Invité" si null ou undefined

```

Différence `??` vs `||` : `||` retourne le fallback pour toute valeur "falsy" (`0`, `""`, `false`). `??` seulement pour `null` / `undefined`.

Modules (import/export)

```

// export nommé
export function foo() {}
export const bar = 42;

// export par défaut (un seul par fichier)
export default function main() {}

// import
import main, { foo, bar } from "./file";
import * as everything from "./file"; // namespace import

```

Async/Await & Promises

```

// Promise

const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("done"), 1000);
});

p.then(result => console.log(result))
  .catch(err => console.error(err));

// Async/await (syntaxe plus lisible)
async function fetchData() {
  try {
    const response = await fetch("/api/data");
    const data = await response.json();
    return data;
  } catch (err) {
    console.error(err);
  }
}

// Promise.all - parallèle
const [a, b, c] = await Promise.all([
  fetchA(),
  fetchB(),
  fetchC(),
]);

// Promise.allSettled - même si un plante, on continue
const results = await Promise.allSettled([...]);

```

Méthodes de tableaux essentielles

```
[1, 2, 3].map(n => n * 2);           // [2, 4, 6]
[1, 2, 3].filter(n => n > 1);        // [2, 3]
[1, 2, 3].reduce((acc, n) => acc + n, 0); // 6
[1, 2, 3].find(n => n === 2);        // 2
[1, 2, 3].some(n => n > 2);          // true
[1, 2, 3].every(n => n > 0);         // true
[1, 2, 3].includes(2);              // true
[1, 2, 3].slice(0, 2);              // [1, 2] (non mutant)
[1, 2, 3].splice(0, 1);             // supprime, mutant
[3, 1, 2].sort((a, b) => a - b);    // [1, 2, 3]
```

Set (utilisé dans MovieActions)

```
const s = new Set([1, 2, 3]);
s.add(4);           // Set { 1, 2, 3, 4 }
s.has(2);           // true
s.delete(1);        // true, retire 1
s.size;             // 3
```


2. TypeScript

Types de base

```
let name: string = "Rayane";
let age: number = 20;
let isActive: boolean = true;
let scores: number[] = [10, 20, 30];
let tuple: [string, number] = ["age", 20];
let anything: any = "évite au max";
let unknown: unknown = "safer que any";
let nothing: void = undefined;
let never: never; // fonctions qui throw ou boucle infinie
```

Interfaces vs Type aliases

```
// Interface (extensible, meilleur pour les objets)
interface User {
  id: string;
  email: string;
  name?: string; // optionnel
}

interface Admin extends User {
  role: "admin";
}

// Type alias (peut être union, intersection, primitif)
type Status = "pending" | "active" | "banned";
type Point = { x: number; y: number };
type UserWithStatus = User & { status: Status };
```

Règle simple : `interface` pour les objets qu'on peut étendre, `type` pour les unions, intersections, ou compositions complexes.

Union et intersection

```
// Union : soit l'un, soit l'autre
type Result = "success" | "error";
type ID = string | number;

// Intersection : les deux à la fois
type Employee = Person & Worker;
```

Generics

```
// Fonction générique
function identity<T>(value: T): T {
    return value;
}

identity<string>("hello"); // T = string
identity(42);              // T = number (inféré)

// Utilisé dans lib/tmdb.ts :
async function fetchTMDB<T>(endpoint: string): Promise<T> {
    const response = await fetch(`${BASE_URL}${endpoint}`);
    return response.json() as Promise<T>;
}

const movie = await fetchTMDB<Movie>("/movie/550");
```

Utility types (à retenir)

```
type User = { id: string; name: string; email: string };

Partial<User>      // Toutes les props optionnelles
Required<User>     // Toutes les props obligatoires
Readonly<User>     // Immutable
Pick<User, "id">    // { id: string }
Omit<User, "email"> // { id, name }
Record<string, number> // { [key: string]: number }
```

Enums

```
enum MovieStatus {
    FAVORITE = "FAVORITE",
    WATCHED = "WATCHED",
    TO_WATCH = "TO_WATCH",
}

// Usage
const status: MovieStatus = MovieStatus.FAVORITE;
```

Note : Prisma génère automatiquement les enums TypeScript à partir du schema.

Type narrowing

```
function process(value: string | number) {
    if (typeof value === "string") {
        value.toUpperCase(); // TS sait que c'est un string
    } else {
        value.toFixed(2);    // TS sait que c'est un number
    }
}
```

as , as const , satisfies

```
const x = "hello" as string;           // Type assertion (à éviter, souvent triche)
const colors = ["red", "blue"] as const; // Type est ["red", "blue"] (tuple readonly)

const config = {
  theme: "dark",
} satisfies { theme: string }; // Vérifie la conformité sans élargir le type
```

3. React 19

Composant fonctionnel

```
type Props = {
  name: string;
  age?: number;
};

function Hello({ name, age = 0 }: Props) {
  return <h1>Bonjour {name}, {age} ans</h1>;
}

// Usage
<Hello name="Rayane" age={20} />
```

JSX

```
// C'est du JS déguisé en HTML
<div className="p-4">          {/* className, pas class */}
  <p>Texte {variable}</p>      {/* {} pour interpoler */}
  {isVisible && <span>Ok</span>} {/* Conditionnel court */}
  {items.map(i => <li key={i.id}>{i.name}</li>)}
</div>
```

Règle absolue : chaque enfant d'une liste doit avoir une prop `key` unique.

useState

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(c => c + 1)}>+ (fonction)</button>
    </>
  );
}
```

Astuce : `setCount(c => c + 1)` (updater function) est plus safe si tu fais plusieurs updates rapides.

useEffect

```
import { useEffect } from "react";

function Component() {
  // Exécuté après chaque render
  useEffect(() => {
    console.log("rendered");
  });

  // Exécuté une seule fois au mount (tableau vide)
  useEffect(() => {
    fetchData();
  }, []);

  // Exécuté quand `id` change
  useEffect(() => {
    fetchUser(id);
  }, [id]);

  // Avec cleanup (utilisé dans TrailerModal)
  useEffect(() => {
    document.body.style.overflow = "hidden";
    return () => {
      document.body.style.overflow = ""; // cleanup au démontage
    };
  }, []);
}
```

useTransition (utilisé dans MovieActions)

```
import { useTransition } from "react";

function MovieActions() {
  const [isPending, startTransition] = useTransition();

  function handleClick() {
    startTransition(async () => {
      await serverAction(); // Long, mais UI reste responsive
    });
  }

  return <button disabled={isPending}>Ajouter</button>;
}
```

Suspense

```
import { Suspense } from "react";

<Suspense fallback={<div>Loading...</div>}>
  <ComponentThatFetches />
</Suspense>
```

Utilisé dans Cinery pour wrapper `SearchBar` (qui utilise `useSearchParams`) sinon Next.js plante au build.

Différences Server vs Client Components

Server Component (défaut)	Client Component
Rendu sur le serveur	Rendu dans le navigateur
Peut être async	Doit être synchrone (top-level)
Peut accéder à la BDD, fs	Peut utiliser useState, useEffect
Pas d'interactivité (pas d'onClick)	Interactif
Directives : rien (défaut)	Directive : <code>"use client"</code> au top

```
// Server component (défaut)
export default async function Page() {
  const data = await prisma.user.findMany();
  return <div>{data.map(...)}</div>;
}

// Client component
"use client";
import { useState } from "react";
export default function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>{n}</button>;
}
```

4. Next.js 16 (App Router)

Structure de fichiers

```
app/
├─ layout.tsx      ← Layout racine (obligatoire)
├─ page.tsx        ← Page d'accueil (/)
├─ globals.css
├─ movie/
│   └─ [id]/
│       └─ page.tsx ← Route dynamique (/movie/123)
├─ search/
│   └─ page.tsx    ← /search
├─ lists/
│   └─ page.tsx    ← /lists
└─ api/
    └─ auth/
        └─ [...nextauth]/
            └─ route.ts ← API route (catch-all)
```

Fichiers spéciaux : - `layout.tsx` : layout partagé - `page.tsx` : la page (l'URL) - `loading.tsx` : UI pendant le chargement (via Suspense) - `error.tsx` : UI en cas d'erreur - `not-found.tsx` : 404 - `route.ts` : endpoint API (au lieu de `page.tsx`) - `middleware.ts` : middleware global

Routes dynamiques

```
// app/movie/[id]/page.tsx
type PageProps = {
  params: Promise<{ id: string }>;    // Async depuis Next 15+
  searchParams: Promise<{ q?: string }>;
};

export default async function MoviePage({ params, searchParams }: PageProps) {
  const { id } = await params;
  const { q } = await searchParams;
  return <div>Film {id}</div>;
}
```

Metadata (SEO)

```
// Statique
export const metadata: Metadata = {
  title: "Cinery",
  description: "Le cinéma sans limites",
};

// Dynamique
export async function generateMetadata({ params }) {
  const { id } = await params;
  const movie = await getMovie(id);
  return {
    title: movie.title,
    description: movie.overview,
    openGraph: {
      images: [movie.posterUrl],
    },
  };
}
```

Data fetching (fetch natif)

```
// Cache par défaut (SSG)
const res = await fetch("https://api.example.com/data");

// Revalidate toutes les 3600s (1h)
const res = await fetch(url, { next: { revalidate: 3600 } });

// Pas de cache (SSR à chaque requête)
const res = await fetch(url, { cache: "no-store" });

// Tag pour invalidation manuelle
const res = await fetch(url, { next: { tags: ["movies"] } });
// Puis : revalidateTag("movies");
```

Server Actions

```
// lib/actions.ts

"use server";

import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

export async function toggleFavorite(movieId: number) {
  const session = await auth();
  if (!session) throw new Error("Not authenticated");

  await prisma.userMovie.create({
    data: { userId: session.user.id, movieId }
  });

  revalidatePath("/lists"); // Invalide le cache
  // ou redirect("/lists");
}
```

Usage depuis un composant client :

```
"use client";

import { toggleFavorite } from "@/lib/actions";

function MovieCard({ movieId }) {
  return (
    <button onClick={() => toggleFavorite(movieId)}>
      Favori
    </button>
  );
}
```

Navigation

```
import Link from "next/link";
import { useRouter } from "next/navigation";

// Lien statique (préchargé)
<Link href="/movie/123">Voir le film</Link>

// Programmatique (dans un client component)
const router = useRouter();
router.push("/movie/123");
router.replace("/login"); // Sans historique
router.back();
router.refresh(); // Re-render server components
```

Image optimisée

```
import Image from "next/image";

<Image
  src="/logo.png"
  alt="Logo"
  width={200}
  height={100}
  priority          // Chargée en priorité (LCP)
/>

// Avec fill (rempli le parent)
<div className="relative w-full h-64">
  <Image src="..." alt="..." fill sizes="100vw" className="object-cover" />
</div>
```

Config nécessaire pour images externes :

```
// next.config.ts
module.exports = {
  images: {
    remotePatterns: [
      { protocol: "https", hostname: "image.tmdb.org" },
    ],
  },
};
```

Middleware (le piège de Cinery)

```
// middleware.ts (à la racine)
import { NextResponse } from "next/server";

export function middleware(request: Request) {
  return NextResponse.next();
}

export const config = {
  matcher: ["/(?!api|_next/static).*/"],
};
```

⚠ **Piège** : Le middleware tourne en **edge runtime**, qui ne supporte **pas** Prisma (`fs` module manquant).
Solution : protéger les routes directement dans les Server Components.

5. Tailwind CSS 4

Concept : utility-first

Au lieu d'écrire du CSS :

```
.card { padding: 16px; background: black; border-radius: 8px; }
```

Tu utilises des classes utilitaires directement dans le HTML :

```
<div className="p-4 bg-black rounded-lg">Card</div>
```

Cheatsheet des classes courantes

Layout

```
flex, inline-flex, grid, block, inline, hidden
flex-row, flex-col, flex-wrap
items-center, items-start, items-end      ← align-items
justify-center, justify-between, justify-around
gap-2, gap-4, gap-8                      ← espace entre enfants
grid-cols-2, grid-cols-3, grid-cols-12
```

Spacing (multiples de 4px par défaut : 1 = 4px)

```
p-4      ← padding: 16px
px-4     ← padding-left/right
py-2     ← padding-top/bottom
pt-8     ← padding-top
m-auto   ← margin: auto
mt-2, mb-4, mx-auto
space-y-4 ← space entre enfants verticaux
```

Sizing

```
w-full, w-1/2, w-64, w-auto
h-screen, h-full, h-64
min-h-screen, max-w-7xl
```

Colors (Cinery)

```
bg-black, bg-white, bg-red-500
bg-cinery-bg, bg-cinery-accent  ← nos couleurs custom
text-white, text-neutral-400
border-neutral-800
```

Typography

```
text-xs, text-sm, text-base, text-lg, text-xl, text-2xl, text-4xl
font-thin, font-normal, font-medium, font-semibold, font-bold, font-black
tracking-tight, tracking-wide      ← letter-spacing
leading-tight, leading-relaxed    ← line-height
uppercase, capitalize
text-center, text-left
```

Effects

```
rounded, rounded-lg, rounded-full
shadow, shadow-lg, shadow-2xl
opacity-50, opacity-0
backdrop-blur-md, backdrop-blur-sm
transition-all, transition-colors, duration-300
```

Interactions

```
hover:bg-red-500      ← au hover
focus:ring-2         ← au focus
active:scale-95
disabled:opacity-50
group-hover:opacity-100 ← quand le parent .group est hover
```

Responsive

Par défaut Tailwind est **mobile-first**. Les prefixes s'appliquent **à partir de** la taille :

```
sm: ≥ 640px   (tablette)
md: ≥ 768px
lg: ≥ 1024px  (desktop)
xl: ≥ 1280px
2xl: ≥ 1536px
```

Exemple :

```
<div className="w-full sm:w-1/2 lg:w-1/3">
  {/* Full sur mobile, moitié sur tablette, tiers sur desktop */}
</div>
```

@theme (Tailwind 4)

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-cinery-bg: #090909;
  --color-cinery-accent: #3772ff;
  --font-sans: "Inter", sans-serif;
}
```

Ensuite disponible partout comme `bg-cinery-bg` , `text-cinery-accent` , `font-sans` .

6. PostgreSQL

Types courants

```
INTEGER, BIGINT
DECIMAL(10, 2), NUMERIC
VARCHAR(255), TEXT
BOOLEAN
DATE, TIMESTAMP, TIMESTAMPTZ (avec fuseau)
JSON, JSONB (indexable)
UUID
ARRAY (ex: TEXT[])
```

Table basique

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  name TEXT,
  age INTEGER CHECK (age >= 0),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Contraintes

```
PRIMARY KEY      -- Identifiant unique de la ligne
UNIQUE           -- Valeur unique dans la colonne
NOT NULL         -- Obligatoire
DEFAULT valeur   -- Valeur par défaut
CHECK (condition) -- Contrainte custom
FOREIGN KEY (col) REFERENCES autre_table(id)
```

Relations

```
-- One-to-Many
CREATE TABLE posts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  content TEXT
);

-- Many-to-Many (via table intermédiaire)
CREATE TABLE user_movies (
  user_id UUID REFERENCES users(id),
  movie_id INTEGER,
  status VARCHAR(20),
  PRIMARY KEY (user_id, movie_id, status)
);
```

Requêtes de base

```
-- SELECT

SELECT * FROM users;

SELECT id, name FROM users WHERE age > 18;

SELECT * FROM users ORDER BY created_at DESC LIMIT 10;


-- JOIN

SELECT u.name, p.content
FROM users u
JOIN posts p ON p.user_id = u.id
WHERE u.age > 18;


-- INSERT

INSERT INTO users (email, name) VALUES ('a@b.com', 'Alice');


-- UPDATE

UPDATE users SET name = 'Bob' WHERE id = '...';


-- DELETE

DELETE FROM users WHERE age < 13;


-- Aggregates

SELECT COUNT(*), AVG(age) FROM users;

SELECT status, COUNT(*) FROM user_movies GROUP BY status;
```

Index

```
-- Speed up queries on ces colonnes

CREATE INDEX idx_users_email ON users(email);

CREATE UNIQUE INDEX idx_unique_email ON users(email);
```

À dire : “Un index accélère les SELECT mais ralentit les INSERT/UPDATE. À poser sur les colonnes filtrées ou jointes souvent.”

Transactions

```
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT; -- ou ROLLBACK si erreur
```

7. Prisma 7 (ORM)

Schema (prisma/schema.prisma)

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id          String   @id @default(cuid())
  email       String   @unique
  name        String?
  createdAt   DateTime @default(now())

  // Relations
  posts       Post[]
  accounts    Account[]
}

model Post {
  id          String @id @default(cuid())
  content     String
  userId      String
  user        User   @relation(fields: [userId], references: [id], onDelete: Cascade)
}

enum Role {
  USER
  ADMIN
}
```

Attributs importants

```
@id                // Clé primaire
@unique            // Unique
@default(now())    // Valeur par défaut
@default(cuid())   // ID généré (cuid = plus lisible que UUID)
@default(autoincrement()) // Auto-increment
@relation(fields: [x], references: [y]) // Relation FK
@@unique([col1, col2]) // Unique composé (sur le model)
@@index([col])        // Index
@@map("nom_sql")      // Renommer la table côté SQL
```

Migrations

```
# Créer une migration à partir du schema
npx prisma migrate dev --name init

# Appliquer les migrations en prod
npx prisma migrate deploy

# Reset complet (destructif)
npx prisma migrate reset

# Générer le client TypeScript
npx prisma generate

# Voir/éditer les données dans le navigateur
npx prisma studio

# Introspecter une BDD existante
npx prisma db pull
```

Queries

```

import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

// findUnique - 1 résultat exact (throw si absent avec findUniqueOrThrow)
const user = await prisma.user.findUnique({
  where: { email: "a@b.com" }
});

// findMany - plusieurs résultats
const users = await prisma.user.findMany({
  where: { age: { gt: 18 } },
  orderBy: { createdAt: "desc" },
  take: 10,
  skip: 20,
  select: { id: true, name: true }, // Ne renvoie que ces champs
  include: { posts: true }, // Inclut la relation
});

// create
const user = await prisma.user.create({
  data: { email: "a@b.com", name: "Alice" }
});

// update
await prisma.user.update({
  where: { id: "..." },
  data: { name: "Bob" }
});

// delete
await prisma.user.delete({ where: { id: "..." } });

// upsert (update OR insert)
await prisma.user.upsert({
  where: { email: "a@b.com" },
  update: { name: "Alice" },
  create: { email: "a@b.com", name: "Alice" }
});

// Transaction
const [user, post] = await prisma.$transaction([
  prisma.user.create({ data: {...} }),
  prisma.post.create({ data: {...} }),
]);

```

Opérateurs de filtre

```
where: {
  age: { gt: 18 },           // greater than
  age: { gte: 18 },          // greater or equal
  age: { lt: 65 },           // less than
  age: { in: [20, 25, 30] },
  name: { contains: "Rayane" }, // LIKE
  name: { startsWith: "R" },
  email: { endsWith: "@gmail.com" },
  AND: [{ age: { gt: 18 } }, { role: "ADMIN" }],
  OR: [{ role: "ADMIN" }, { role: "MODERATOR" }],
  NOT: { deleted: true },
}
```

Adapter PostgreSQL (Prisma 7)

Prisma 7 impose un **adapter** pour se connecter :

```
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";
import { Pool } from "pg";

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

const adapter = new PrismaPg(pool);
const prisma = new PrismaClient({ adapter });
```

8. Auth.js v5 & OAuth 2.0

OAuth 2.0 — le flow

1. User clique "Se connecter avec Google"
2. Notre app redirige vers Google : `/oauth/authorize?client_id=...&redirect_uri=...`
3. User autorise sur Google
4. Google redirige vers notre callback avec un `?code=xxx`
5. Notre serveur échange le code contre un `access_token` + user info
6. On crée/retrouve l'utilisateur en BDD, on démarre une session

Termes : - **Client ID** : identifiant public de notre app chez Google - **Client Secret** : mot de passe confidentiel de notre app (jamais exposé côté client) - **Redirect URI** : où Google doit renvoyer après auth (`/api/auth/callback/google`) - **Scopes** : ce qu'on demande (email, profile, etc.)

Setup Auth.js v5

```
// lib/auth.ts

import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import { PrismaAdapter } from "@auth/prisma-adapter";

export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(prisma),
  providers: [
    Google({
      clientId: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
    }),
  ],
  session: {
    strategy: "database", // ou "jwt"
  },
  pages: {
    signIn: "/login",
  },
});
```

```
// app/api/auth/[...nextauth]/route.ts

export { GET, POST } from "@lib/auth";
```

Session strategies

database	jwt
Session en BDD	Session dans un cookie signé
Requête BDD à chaque page	Pas de requête BDD
Peut être révoquée	Impossible à révoquer (jusqu'à expiration)
Plus lent	Plus rapide

Utilisation

```
// Server component
import { auth } from "@/lib/auth";

export default async function Page() {
  const session = await auth();
  if (!session) redirect("/login");
  return <p>Bonjour {session.user.name}</p>;
}

// Server action pour se connecter/déconnecter
async function login() {
  "use server";
  await signIn("google");
}

async function logout() {
  "use server";
  await signOut({ redirectTo: "/" });
}
```

Tables requises par PrismaAdapter (noms exacts)

```
model User { ... }
model Account { ... }           // Comptes OAuth liés
model Session { ... }           // Sessions actives
model VerificationToken { ... } // Tokens temporaires
```

9. HTTP / REST / API

Méthodes HTTP

Verbe	Usage	Idempotent	Safe
GET	Lire	✓	✓
POST	Créer	✗	✗
PUT	Remplacer complet	✓	✗
PATCH	Modifier partiel	✗	✗
DELETE	Supprimer	✓	✗

Idempotent = plusieurs appels = même résultat. **Safe** = ne modifie pas la ressource.

Codes de statut

Code	Signification
200	OK
201	Created
204	No Content (delete réussi)
301	Moved Permanently
302	Found (redirection temporaire)
303	See Other (redirection après POST)
304	Not Modified (cache valide)
400	Bad Request
401	Unauthorized (pas authentifié)
403	Forbidden (authentifié mais pas autorisé)
404	Not Found
409	Conflict
422	Unprocessable Entity
429	Too Many Requests (rate limit)
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

Headers courants

```
Content-Type: application/json
Authorization: Bearer eyJhbGc...
Accept: application/json
Cache-Control: no-cache, max-age=3600
Cookie: sessionToken=abc123
Set-Cookie: sessionToken=xyz; HttpOnly; Secure; SameSite=Lax
```

Fetch API (native)

```
// GET
const res = await fetch("https://api.example.com/movies");
const data = await res.json();

// POST
const res = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`,
  },
  body: JSON.stringify({ email: "a@b.com" }),
});

if (!res.ok) throw new Error(`HTTP ${res.status}`);
```

Bearer token (TMDB v4)

```
const response = await fetch("https://api.themoviedb.org/3/movie/popular", {
  headers: {
    "Authorization": `Bearer ${process.env.TMDB_ACCESS_TOKEN}`,
    "Accept": "application/json",
  },
});
```

CORS (Cross-Origin Resource Sharing)

Un navigateur bloque les requêtes JS vers un autre domaine sauf si le serveur autorise via headers :

```
Access-Control-Allow-Origin: https://mon-site.com
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Content-Type, Authorization
```

À dire : “Dans Cinery, TMDB est appelé depuis le serveur Next.js, donc pas de CORS. Si j’appelais depuis le navigateur, il faudrait que TMDB whitelist mon domaine.”

REST vs GraphQL

REST	GraphQL
Plusieurs endpoints	Un seul endpoint

Le serveur décide de la structure	Le client demande exactement ce qu'il veut
Over/under fetching fréquent	Fetch précis
Standard, cache HTTP facile	Plus complexe, mais flexible

10. Git & GitHub

Commandes essentielles

```
# Initialiser
git init
git clone https://github.com/user/repo

# Vérifier l'état
git status
git log --oneline
git diff

# Stage & commit
git add fichier.ts      # Ajouter un fichier
git add .               # Ajouter tout
git commit -m "message" # Commit
git commit -am "msg"    # Add + commit (fichiers déjà tracked)

# Branches
git branch              # Liste
git branch nom          # Créer
git checkout nom        # Changer
git checkout -b nom     # Créer et changer
git merge nom           # Merger nom dans la branche courante

# Remote (GitHub)
git remote -v           # Voir les remotes
git remote add origin URL
git push                # Envoyer sur GitHub
git push -u origin main # Premier push (set upstream)
git pull                # Récupérer les changements
git fetch                # Récupérer sans merger

# Annuler
git restore fichier     # Annuler modifs non stagées
git reset HEAD fichier  # Unstage un fichier
git reset --hard HEAD~1 # Annuler le dernier commit (destructif)
git revert commit-hash  # Créer un commit qui annule un ancien

# Historique
git log --graph --all --oneline
git blame fichier       # Qui a modifié quelle ligne
```

.gitignore

Fichier à la racine qui liste ce que Git doit ignorer :

```
node_modules/  
.env  
.env.local  
.next/  
*.log  
.DS_Store
```

Workflow typique

```
# 1. Récupérer les derniers changements  
git pull origin main  
  
# 2. Créer une branche pour la feature  
git checkout -b feature/auth  
  
# 3. Bosser, commit régulièrement  
git add .  
git commit -m "Add Google OAuth setup"  
  
# 4. Push la branche  
git push -u origin feature/auth  
  
# 5. Créer une Pull Request sur GitHub  
# 6. Review, merger dans main  
# 7. Retour local :  
git checkout main  
git pull  
git branch -d feature/auth
```

11. Vercel & Neon (déploiement)

Vercel

Ce que c'est : Plateforme d'hébergement optimisée pour Next.js. Serverless, CDN mondial, CI/CD auto.

Comment ça marche : 1. Tu connectes ton repo GitHub à Vercel 2. À chaque `git push`, Vercel : - Clone le repo - Lance `npm install` - Lance `npm run build` - Déploie sur `<projet>.vercel.app` 3. Si le build échoue → l'ancienne version reste en ligne

Serverless Functions : chaque API route / Server Action tourne dans une fonction éphémère qui démarre à la demande.

Environments : - **Production** : branche `main`, URL principale - **Preview** : autres branches / PR, URL unique par branche - **Development** : local

Env vars : chaque variable (`DATABASE_URL`, etc.) doit être ajoutée dans Settings → Environment Variables, sinon le build utilise `undefined`.

Neon

Ce que c'est : PostgreSQL managed serverless. Base cloud, plan gratuit généreux.

Pourquoi ? Vs SQLite : - Vercel a un filesystem éphémère (recréé à chaque build) - Vercel filesystem est en read-only en runtime - Donc SQLite (fichier local) est **inutilisable** en prod - Solution : BDD réseau → PostgreSQL sur Neon

Connection pooling : Neon fournit deux URLs : - **Direct** : connexion classique - **Pooled** (avec `-pooler` dans le hostname) : mieux pour serverless (chaque fonction peut créer une conn)

Runtime Node vs Edge

Node runtime	Edge runtime
Node.js complet	Sous-ensemble V8
Peut utiliser fs, crypto, Prisma	Non
Démarrage plus lent	Démarrage instantané
Défaut Vercel	Middleware, edge functions

Piège Cinery : le middleware Next.js est en edge runtime, ne peut pas utiliser Prisma. Solution : protéger les pages dans les Server Components.

12. Node.js

Modules

```
// CommonJS (ancien)

const fs = require("fs");
module.exports = { foo };

// ESM (moderne, utilisé partout maintenant)

import fs from "fs";
export function foo() {}
```

Async/await + Promises

```
// Callback (ancien, à éviter)
fs.readFile("file.txt", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});

// Promise
fs.promises.readFile("file.txt")
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Async/await
try {
  const data = await fs.promises.readFile("file.txt");
} catch (err) {
  console.error(err);
}
```

npm / package.json

```
# Installer les deps du package.json
npm install

# Installer une nouvelle dep
npm install express

# Installer en dev
npm install -D typescript

# Installer globalement
npm install -g vercel

# Lancer un script du package.json
npm run dev
npm run build
```

package.json :

```
{
  "name": "cinery",
  "version": "0.1.0",
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "postinstall": "prisma generate"
  },
  "dependencies": {
    "next": "^16.2.9",
    "react": "^19.0.0"
  },
  "devDependencies": {
    "typescript": "^5.0.0"
  }
}
```

Différence : - `dependencies` : nécessaires en prod - `devDependencies` : uniquement pour développer (TypeScript, ESLint, tests)

Variables d'environnement

```
// Charger depuis .env
process.env.DATABASE_URL
process.env.NODE_ENV // "development" ou "production"

// Avec dotenv (chargé automatiquement par Next)
import "dotenv/config";
```

13. Sécurité web (OWASP Top 10)

1. XSS (Cross-Site Scripting)

Problème : l'attaquant injecte du JS dans une page vue par d'autres users.

```
// ✗ Vulnérable
<div dangerouslySetInnerHTML={{ __html: userInput }} />

// ✔ Safe (React échappe automatiquement)
<div>{userInput}</div>
```

Cinery : React échappe partout, pas de `dangerouslySetInnerHTML`.

2. CSRF (Cross-Site Request Forgery)

Problème : un site malveillant fait une requête à ton site avec les cookies de l'user (transactions bancaires, etc.).

Protection : token CSRF unique dans le formulaire, vérifié côté serveur.

Cinery : Server Actions Next.js ont un **CSRF token natif** vérifié automatiquement. Impossible d'appeler une Server Action depuis un autre domaine.

3. SQL Injection

Problème : l'user injecte du SQL via un input.

```
// ✗ Vulnérable
const query = `SELECT * FROM users WHERE email = '${userEmail}'`;
// Si userEmail = "'; DROP TABLE users; --" → catastrophe

// ✔ Safe (prepared statement)
const query = "SELECT * FROM users WHERE email = $1";
db.query(query, [userEmail]);
```

Cinery : Prisma utilise des prepared statements pour toutes les queries. Impossible d'injecter du SQL.

4. IDOR (Insecure Direct Object Reference)

Problème : l'user accède à des données qui ne lui appartiennent pas en changeant un ID dans l'URL.

```
// ✗ Vulnérable
GET /api/orders/123 → renvoie la commande 123, peu importe qui la demande

// ✔ Safe
GET /api/orders/123
→ vérifier que session.user.id === order.userId avant de renvoyer
```

Cinery : toutes les Server Actions scopent par `userId` extrait de la session serveur.

5. Authentication broken

Problèmes : mots de passe faibles stockés en clair, sessions qui n'expirent pas, etc.

Cinery : - OAuth Google → **aucun mot de passe stocké** - Sessions expirent (30 jours par défaut Auth.js) -
Peuvent être révoquées (session en BDD)

6. Exposition de secrets

```
// ✗ NE JAMAIS commit  
const API_KEY = "sk_live_abc123";  
  
// ✔ Utiliser les env vars  
const API_KEY = process.env.API_KEY;
```

Règles Cinery : - `.env` et `.env.local` dans `.gitignore` - Vercel env vars marquées "Sensitive" - Rotation régulière des secrets

7. Autres vulnérabilités à connaître

- **SSRF** (Server-Side Request Forgery) : le serveur fait une requête à une URL contrôlée par l'attaquant
- **Path Traversal** : `?file=../../etc/passwd`
- **Race Conditions** : deux actions simultanées créent un état incohérent
- **Rate limiting absent** : attaquant peut brute-force
- **CVE** : Common Vulnerabilities and Exposures — DB publique de failles connues

Headers de sécurité

```
Strict-Transport-Security: max-age=31536000 ← HTTPS forcé  
X-Frame-Options: DENY ← anti-clickjacking  
X-Content-Type-Options: nosniff  
Content-Security-Policy: default-src 'self' ← whitelist des sources  
Referrer-Policy: no-referrer
```

Cookies

```
Set-Cookie: sessionId=abc; HttpOnly; Secure; SameSite=Lax; Max-Age=2592000
```

- **HttpOnly** : inaccessible en JS (protège du XSS)
 - **Secure** : HTTPS seulement
 - **SameSite=Lax/Strict** : anti-CSRF
 - **Max-Age** : durée en secondes
-

14. Terminal Bash

Navigation

```
pwd          # Où suis-je ?
ls           # Lister
ls -la       # Lister avec détails + cachés
cd ~/Bureau  # Aller à
cd ..        # Remonter
cd -         # Revenir au dossier précédent
```

Fichiers

```
cat fichier          # Afficher
less fichier         # Afficher paginé (q pour quitter)
head -20 fichier     # 20 premières lignes
tail -20 fichier     # 20 dernières
tail -f fichier      # Suivre en direct
wc -l fichier        # Compter les lignes

mkdir dossier        # Créer un dossier
mkdir -p a/b/c       # Créer récursivement
touch fichier        # Créer un fichier vide
cp source dest       # Copier
cp -r src/ dest/     # Copier récursif
mv old new           # Renommer/déplacer
rm fichier           # Supprimer
rm -rf dossier       # Supprimer récursivement (danger)
```

Recherche

```
grep "texte" fichier      # Chercher dans un fichier
grep -r "texte" dossier/  # Récursif
grep -i "texte" fichier   # Insensible à la casse
grep -n "texte" fichier   # Avec numéro de ligne

find . -name "*.ts"       # Trouver fichiers
find . -type d            # Trouver dossiers
find . -mtime -7          # Modifiés dans les 7 derniers jours
```

Redirections

```
cmd > fichier           # Rediriger stdout (écrase)
cmd >> fichier           # Ajouter à la fin
cmd 2> erreurs.log      # Rediriger stderr
cmd &> tout.log          # Rediriger tout
cmd < fichier            # Lire depuis un fichier

# Heredoc
cat > fichier << 'EOF'
contenu
sur plusieurs lignes
EOF
```

Pipes

```
ls | grep ".ts"         # Filtrer la sortie
ps aux | grep node      # Chercher un process
cat file | wc -l        # Compter lignes
history | tail -20      # 20 dernières commandes
```

Autre

```
sudo cmd                # Exécuter en root
chmod +x script.sh      # Rendre exécutable
export VAR=value        # Définir env var temporaire
which node              # Trouver le path d'un binaire
node --version          # Version
Ctrl+C                 # Interrompre
Ctrl+D                 # Fin de fichier / logout
Ctrl+L                 # Clear screen
```

15. Concepts transverses

Client-side vs Server-side rendering

CSR	SSR	SSG	ISR
HTML vide, JS render tout	HTML rendu par le serveur	HTML pré-généré au build	SSG + revalidate régulier
Lent premier chargement	Rapide, SEO OK	Ultra-rapide, mais statique	Compromis
React classique (Vite)	Next.js Server Component	Next.js generateStaticParams	Next.js revalidate: 3600

Cinery : mix SSR (Server Components) + ISR (revalidate 1h sur TMDB).

Cache

- **Browser cache** : cookies, localStorage, cache HTTP
- **CDN cache** : Vercel, Cloudflare — cache les assets statiques mondialement
- **Server cache** : Next.js cache `fetch()`
- **Database cache** : Redis, Memcached (Cinery n'en a pas)

Async / synchrone

```
// Synchrone (bloquant)

const data = readFileSync("file.txt");
console.log(data);
console.log("après"); // Attend que la lecture soit finie

// Asynchrone (non bloquant)
readFile("file.txt", (err, data) => {
  console.log(data);
});
console.log("après"); // S'exécute AVANT le contenu du fichier
```

Event loop (Node.js)

Node est **single-threaded** mais gère plein de tâches en parallèle grâce à l'évent loop :

1. Une tâche async est envoyée à un thread worker (I/O)
2. Node continue à exécuter le reste du code
3. Quand le worker a fini, il pousse le résultat dans une queue
4. L'évent loop pioche dans la queue et exécute les callbacks

Rate limiting

Limiter le nombre de requêtes par utilisateur/IP par unité de temps : - TMDB : 50 req/sec - GitHub API : 5000 req/h (avec token) - Solutions : Redis + middleware, ou Upstash pour serverless

Env : dev vs prod

```
NODE_ENV=development  # Local
NODE_ENV=production   # Vercel
NODE_ENV=test         # Tests
```

Utile pour : - Charger `.env.local` en dev, `.env.production` en prod - Activer les logs verbose en dev - Optimiser le bundle en prod

🎯 Points bonus à retenir

Le “je sais ce que je ne maîtrise pas bien”

C’est une vraie force en entretien de savoir dire : - “Je n’ai pas encore écrit de tests unitaires sur ce projet, ce serait la prochaine étape.” - “Je n’ai pas encore mis en place de monitoring type Sentry.” - “Je n’ai jamais utilisé Docker sur un projet perso, mais je comprends le concept.”

Le vocabulaire pro

- “L’API que **j’ai consommée**” (pas “utilisée”)
- “**J’ai orchestré** les appels avec Promise.all”
- “**J’ai sécurisé** les mutations avec des vérifications de session”
- “**J’ai instrumenté** le code avec des logs pour débbugger”
- “**J’ai pivoté** de SQLite à PostgreSQL pour des raisons de compatibilité serverless”
- “**J’ai itéré** en 6 sprints”

Les 3 questions pièges classiques

“**Explique ce que fait `useEffect`**” : > “C’est un hook React qui exécute une fonction après le rendu du composant. On lui passe un tableau de dépendances : si vide, ça s’exécute une seule fois au mount. Si on met des variables, ça re-s’exécute quand elles changent. On peut aussi retourner une fonction de cleanup qui s’exécute au démontage — utile pour supprimer des listeners, timeouts, etc.”

“**C’est quoi la différence entre `let` et `const` ?**” : > “`const` interdit la réassignation de la variable, mais si c’est un objet ou tableau, on peut toujours modifier ses propriétés. `let` permet de réassigner. Par convention on utilise `const` par défaut.”

“**Pourquoi TypeScript et pas JavaScript ?**” : > “TypeScript apporte le typage statique — les erreurs sont détectées au compile-time au lieu du runtime. C’est particulièrement utile sur des projets avec des APIs externes ou une BDD : quand je change un champ dans le schéma Prisma, TS me signale immédiatement tous les endroits du code à mettre à jour. Ça réduit les bugs et facilite le refactoring.”

Ressources pour approfondir

- **Docs officielles** : la meilleure source
 - React : <https://react.dev>
 - Next.js : <https://nextjs.org/docs>
 - Prisma : <https://www.prisma.io/docs>
 - Tailwind : <https://tailwindcss.com/docs>
 - MDN (JS/TS) : <https://developer.mozilla.org>
- **Playgrounds** :
 - TypeScript : <https://www.typescriptlang.org/play>
 - React : <https://codesandbox.io>
 - SQL : <https://sqlbolt.com>
- **Certifications gratuites** (pour ton CV) :
 - freeCodeCamp
 - Odin Project
 - LinkedIn Learning (via bibliothèque universitaire)

Fin du mémo. Bonne chance en entretien 